

RELATÓRIO TRABALHO FINAL

Sistema Cloud Elástico para Detecção e Tradução de Características em Imagens

CN2026Labels

U.C.: Computação na Nuvem

PROJETO: CN2026Labels

ALUNOS: Daniel Santos, 32078
João Galveia, 50873
Ricardo Nobre, 48713

ANO LETIVO: 2025/2026

Índice

1. Introdução	2
1.1. Enquadramento do problema	2
1.2. Âmbito do relatório	2
2. Objetivos e requisitos do projeto	3
2.1. Objetivo geral.....	3
2.2. Operações funcionais - SF.....	3
2.3. Operações de gestão de elasticidade - SG	3
2.4. Serviços GCP envolvidos	3
3. Arquitetura da solução	4
3.1. Componentes principais	4
3.2. Fluxo de submissão e processamento	5
3.3. Elasticidade.....	5
4. Modelo de dados e formatos de mensagens	6
4.1. Identificador do pedido	6
4.2. Cloud Storage	6
4.3. Mensagem Pub/Sub	6
4.4. Firestore	7
4.5. Estados do pedido	7
5. Implementação	8
5.1. Cliente gRPC.....	8
5.2. Servidor gRPC	8
5.3. Labels App.....	8
5.4. Lookup Function	9
5.5. Contrato Protobuf	9
5.6. Tratamento de Erros.....	9
6. Deployment e execução	10
6.1. Recursos GCP criados.....	10
6.2. Execução das aplicações	10
6.3. Configuração necessária.....	10
7. Validação dos Resultados e Objetivos Atingidos	11
7.1. Testes funcionais	11
7.2. Testes de gestão de elasticidade	13
8. Conclusão.....	14

1. Introdução

1.1. Enquadramento do problema

Atualmente, muitas aplicações necessitam de processar imagens de forma automática, identificando o seu conteúdo e organizando-as com base nessa informação. No entanto esse processamento pode ser demorado, especialmente quando envolve operações como deteção de objetos, classificação por *labels* e tradução de resultados.

Este trabalho propõe um sistema distribuído na Google Cloud Platform (GCP) que permite a aplicações clientes submeter ficheiros de imagem, detetar automaticamente o seu conteúdo e classificá-los através da atribuição de *labels* (ex: cat, tree, car). As *labels* detectadas são posteriormente traduzidas para Português.

1.2. Âmbito do relatório

Este relatório descreve a arquitetura e implementação do sistema, incluindo a submissão de imagens através de *gRPC*, o armazenamento em *Cloud Storage*, o processamento assíncrono com *Pub/Sub* e *workers*, e a persistência de resultados em *Firestore*.

O sistema suporta elasticidade, permitindo aumentar o número de servidores *gRPC* e de *workers* consoante a carga de trabalho.

2. Objetivos e requisitos do projeto

2.1. Objetivo geral

Desenvolver o sistema *CN2026Labels* para submissão, processamento assíncrono e consulta de resultados de análise de imagens, com elasticidade gerida via *RPCs* dedicadas.

2.2. Operações funcionais - SF

- Submissão de uma imagem através de streaming *gRPC*, guardando o ficheiro como *blob* no *Cloud Storage*.
- Devolução de um identificador único do pedido, usado posteriormente para consultar o resultado.
- Consulta das *labels* detetadas, respetivas traduções e data de processamento.
- Pesquisa de ficheiros armazenados entre duas datas que contenham uma determinada característica.
- Operações opcionais implementadas, caso existam, como *download* de uma imagem encontrada, e listagem das imagens já submetidas.

2.3. Operações de gestão de elasticidade - SG

- Aumento e diminuição do número de instâncias dos servidores *gRPC*.
- Aumento e diminuição do número de instâncias da aplicação de processamento de imagens.

2.4. Serviços GCP envolvidos

Serviço	Funções no sistema
Cloud Storage	Armazenamento dos <i>blobs</i> de imagem submetidos.
Firestore (nativo)	Armazenamento dos dados de processamento (<i>labels</i> , traduções e confiança).
Cloud Pub/Sub	Fila assíncrona em <i>work-queue pattern</i> entre servidor <i>gRPC</i> e <i>Labels App</i>
Compute Engine (MIGs)	Dois <i>MIGs</i> : um para servidores <i>gRPC</i> , outro para <i>workers Labels App</i> .
Cloud Run Functions	Função <i>HTTP</i> de <i>lookup</i> dos <i>IPs</i> do <i>MIG gRPC</i> .
Vision API	Deteção de <i>labels</i> , com pontuação de confiança 0 ... 1.
Translation API	Tradução das <i>labels</i> de inglês para português.

3. Arquitetura da solução

A arquitetura do sistema encontra-se dividida em vários componentes independentes, cada um responsável por uma função específica dentro da plataforma. Esta separação permite uma maior modularidade, escalabilidade e facilidade de manutenção, assegurando simultaneamente uma comunicação eficiente entre os diferentes serviços distribuídos.

3.1. Componentes principais

- **Cliente gRPC** – Interface CLI utilizada pelos utilizadores para comunicar com o sistema.
- **Servidor gRPC** – Componente central responsável por expor os serviços remotos e coordenar os pedidos recebidos.
- **Serviço Funcional (SFService)** – Responsável pelas operações sobre imagens, como submissão, pesquisa e download.
- **Serviço de Escalabilidade (SGService)** – Responsável pela gestão dinâmica da infraestrutura.
- **LabelsApp / Processor** – Executa o processamento assíncrono de imagens e deteção de *labels*.
- **Lookup Function** – Disponibiliza dinamicamente os endereços IP dos servidores disponíveis.
- **Cloud Storage** – Serviço utilizado para armazenamento persistente das imagens.
- **Firestore** – Base de dados utilizada para guardar metadados e resultados de processamento.
- **Pub/Sub** – Sistema de mensagens utilizado para comunicação assíncrona entre módulos.
- **Managed Instance Groups (MIGs)** – Permitem a escalabilidade horizontal da infraestrutura.
- **Contrato Protobuf** – Define os serviços, métodos e mensagens utilizados na comunicação *gRPC*.

3.2. Fluxo de submissão e processamento

Esta secção descreve o fluxo completo de submissão e processamento de imagens no sistema, desde o envio realizado pelo cliente até ao armazenamento dos resultados finais. São apresentados os principais passos envolvidos, bem como a interação entre os diferentes componentes e serviços distribuídos da plataforma. Este fluxo segue então os seguintes passos:

- O cliente obtém a lista de servidores *gRPC* através da *Lookup Function*.
- Escolhe posteriormente um desses servidores e submete a imagem por streaming.
- O servidor guarda a imagem no *Cloud Storage* e publica uma mensagem no *Pub/Sub*.
- A *Labels App* consome a mensagem, obtém a imagem, deteta *labels* e traduz os resultados.
- Os resultados são guardados no *Firestore*.
- O cliente consulta posteriormente o resultado usando o identificador do pedido, ou o nome da imagem.

Vale mencionar que o processamento é totalmente assíncrono após o *upload*.

3.3. Elasticidade

O sistema escala em duas dimensões independentes - servidores *gRPC* e *workers Labels App*, pois os perfis de carga são diferentes:

- O servidor *gRPC* é maioritariamente *I/O-bound* (*uploads*, escrita em *Storage*, leituras *Firestore*).
- O *worker* é *CPU/API-bound* (espera *Vision* e *Translate* cerca de 1 a 3 segundos por imagem).

Esta diferença justifica a utilização de dois *MIGs* distintos: um para os servidores *gRPC* e outro para os *workers* da *Labels App*. Usar apenas um *MIG* iria acoplar a escala dos dois serviços, expor os *IPs* dos *workers* no *lookup* e aumentar o impacto de falhas entre processos.

4. Modelo de dados e formatos de mensagens

4.1. Identificador do pedido

`requestId = UUID.randomUUID().toString()`.

É gerado pelo servidor *gRPC* no fim do *submitImage* e é simultaneamente:

- A chave *Firestore* (*docId*)
- O prefixo do caminho no *bucket* (*<requestId>/<imageName>*),
- Um atributo da mensagem *Pub/Sub* (para filtros sem desserializar o *JSON*)

4.2. Cloud Storage

- **Bucket:** *cn2526-t3-g07-trab-final* (região *europa-west1*).
- **Nome do *blob*:** *<requestId>/<imageName>*.
- **Metadados do *blob*:** *Content-Type* original do ficheiro e *metadata* da imagem.

4.3. Mensagem Pub/Sub

- **Tópico:** *cn2026labels-processing*.
- **Subscrição compartilhada:** *cn2026labels-processing-sub*.
- **Data:** (*UTF-8 JSON*):

```
{
  "requestId": "<uuid>",
  "gsUri":      "gs://cn2526-t3-g07-trab-
final/<requestId>/<imageName>",
  "imageName": "<imageName>",
  "bucket":    "cn2526-t3-g07-trab-final"
}
```

Para além do *data*, a mensagem carrega um atributo *Pub/Sub* *requestId* com o mesmo valor. Esta duplicação permite filtros do lado da *Pub/Sub* (que operam sobre atributos, não sobre o *payload*) e facilita a inspeção na consola *GCP*.

JSON foi escolhido em vez de *protobuf* binário por ser legível na consola *GCP* e por evitar partilha extra do *.proto* com o *worker*. O tamanho da mensagem é pequeno; o custo de serialização não é mensurável face às chamadas *Vision/Translate*.

4.4. Firestore

- **Base de dados:** *trab-final-db* (modo nativo).
- **Coleção:** *image_processing*.
- **docId** = *requestId* (UUID).

Campo	Tipo	Quando é escrito	Comentário
request_id	string	UPLOADED + PROCESSED	Igual ao <i>docId</i> ; redundante
image_name	string	UPLOADED	Nome original do ficheiro
content_type	string	UPLOADED	image/jpg
bucket_name	string	UPLOADED + PROCESSED	Nome do <i>bucket</i>
size_bytes	long	UPLOADED	Tamanho do blob recebido
status	string	UPLOADED -> PROCESSED	Máquina de estados de duas fases
uploaded_at	timestamp	UPLOADED	Fim do <i>submitImage</i>
processed_at	timestamp	PROCESSED	Fim do processamento do <i>worker</i>
labels_eng	array	PROCESSED	Devolvidas pelo <i>Vision</i>
labels_pt	array	PROCESSED	Traduções alinhadas por índice
labels_confidence	array	PROCESSED	Score 0..1 alinhado por índice
labels	array	PROCESSED	{label_eng, label_pt, confidence_score} Para getImageData
labels_count	int	PROCESSED	Tamanho dos arrays

4.5. Estados do pedido

O ciclo de vida tem **duas fases** explícitas:

- **UPLOADED** - Estado após o *submitImage* terminar com sucesso. O blob encontra-se no Storage, o documento foi criado e a mensagem foi publicada pelo Pub/Sub.
- **PROCESSED** - Estado após o *LabelsApp* escrever no Firestore. Está disponível toda a informação do estado anterior para além das labels e o campo "processed_at".

5. Implementação

O código está em cinco módulos Maven: *cn2026contract* (*proto*), *cn2026server* (*gRPC* + *scaling*), *cn2026labels* (*ProcessorMain* + *helpers*), *cn2026client* (*CLI*) e *cn2026lookupfunction* (Google Cloud Run Function). Existe também *vision-translation* como *playground* original, não usado no *deployment*. Cada módulo gera um JAR através do *maven-assembly-plugin*. Foi necessário usar um *assembly.xml* para combinar os ficheiros *META-INF/services* do *gRPC*. Caso contrário, o *Firestore* falhava ao arrancar.

5.1. Cliente gRPC

O módulo cliente implementa uma interface de linha de comandos (*CLI*) responsável por comunicar com os serviços *gRPC* do sistema. Este permite ao utilizador executar operações funcionais sobre imagens, como submissão, pesquisa, listagem e download, bem como operações de gestão e elasticidade da infraestrutura, incluindo o aumento ou redução do número de servidores *gRPC* e instâncias de processamento. O cliente obtém dinamicamente os servidores disponíveis através de uma *Cloud Function*, estabelece a ligação *gRPC* ao servidor selecionado e encaminha os pedidos para os respetivos serviços através de stubs bloqueantes e não bloqueantes.

5.2. Servidor gRPC

O módulo servidor implementa a infraestrutura central do sistema, usando dois serviços *gRPC* distintos: um serviço funcional, responsável pelas operações sobre imagens, e um serviço de escalabilidade, responsável pela gestão dinâmica da infraestrutura. O servidor inicia as ligações ao *Firestore*, ao *Google Cloud Storage* e ao *Pub/Sub*, permitindo armazenar imagens, guardar metadados e publicar pedidos de processamento assíncrono. Além disso, integra mecanismos de elasticidade através de *Managed Instance Groups (MIGs)*, possibilitando aumentar ou reduzir dinamicamente o número de servidores *gRPC* e instâncias de processamento. O serviço funcional disponibiliza operações como submissão, pesquisa, listagem e download de imagens, enquanto o serviço de escalabilidade gere o redimensionamento seguro da infraestrutura, incluindo validações e confirmações para operações críticas, como a redução do número de instâncias para zero.

5.3. Labels App

O módulo *LabelsApp* é responsável pelo processamento assíncrono das imagens submetidas ao sistema. Este módulo recebe mensagens publicadas no *Pub/Sub*, executa deteção automática de *labels* através da *Google Cloud Vision API* e traduz os resultados para português utilizando a *Google Translate API*. Após o processamento, os metadados e *labels* identificados são armazenados no *Firestore*, incluindo informações como níveis de confiança, estado do processamento e *timestamps*. O módulo foi desenvolvido como um processo independente, permitindo desacoplar o processamento de imagens do servidor *gRPC* principal e facilitar a escalabilidade das instâncias de processamento.

5.4. Lookup Function

O módulo *LookupFunction* implementa uma *Google Cloud Function* responsável por disponibilizar dinamicamente os endereços IP das instâncias ativas do grupo de servidores *gRPC*. A função consulta o *Managed Instance Group (MIG)* através da *Compute Engine API*, obtém as instâncias atualmente em execução e extrai os respectivos endereços IP públicos. Estes são devolvidos ao cliente sob a forma de uma resposta *HTTP* em formato *JSON*, permitindo que o cliente descubra automaticamente servidores disponíveis sem necessidade de configuração manual de endereços.

5.5. Contrato Protobuf

O contrato *Protobuf* define a interface de comunicação entre clientes e servidores através de *gRPC*, especificando os serviços remotos disponíveis e as estruturas de dados trocadas entre os diferentes módulos do sistema. Este contrato inclui o serviço funcional (*SFService*), responsável pelas operações relacionadas com imagens, como submissão, pesquisa, listagem e download, e o serviço de escalabilidade (*SGService*), responsável pelas operações de gestão da infraestrutura. Define ainda várias mensagens de comunicação utilizadas pelos serviços *gRPC*, como *SubmitImageRequest*, *SearchImagesRequest*, *ScaleRequest* ou *GetImageDataResponse*, permitindo estruturar de forma consistente os pedidos e respostas trocados entre os diferentes módulos do sistema.

5.6. Tratamento de Erros

Dos códigos de erros usados neste projeto, alguns a considerar são:

- **INVALID_ARGUMENT** : argumento inválido, por exemplo, `java -jar GrpcServer.java -1000` (não existem portos negativos);
- **NOT_FOUND** : recurso não encontrado;
- **INTERNAL** : falhas internas do servidor;
- **UNAVAILABLE** : a configuração de *scaler GCP* não está disponível;
- **PERMISSION_DENIED** : o cliente não tem autorização para realizar a operação pretendida;
- **ALREADY_EXISTS** : uma imagem submetida já existe nos serviços da *Google*.

6. Deployment e execução

Esta secção descreve os principais recursos criados na Google Cloud Platform, bem como os procedimentos necessários para execução e configuração das diferentes aplicações do sistema.

6.1. Recursos GCP criados

Recurso	Finalidade
Cloud Storage	Armazenamento das imagens
Firestore Database	Persistência de metadados e resultados
Pub/Sub	Comunicação assíncrona
Managed Instance Groups (Migs)	Criação automática de instâncias
Cloud Run Function	Descoberta dinâmica de IPs
VM Templates	Criação do template para as instâncias
Service Accounts	Autenticação e permissões

6.2. Execução das aplicações

A execução do sistema requer apenas:

- Redimensionamento de ambos os *MIGs* de forma a haver pelo menos 1 instância de *Server* e *LabelsApp* :
 - `gcloud compute instance-groups managed resize cn2026-server-mig --size=1 --zone=europe-west1-b`
 - `gcloud compute instance-groups managed resize cn2026-labels-mig --size=1 --zone=europe-west1-b`
- Execução da aplicação cliente *gRPC*.

As instâncias são iniciadas automaticamente através dos startup scripts configurados nos templates das VMs.

6.3. Configuração necessária

Para executar o sistema é apenas necessário:

- Aplicação *GrpcClient*;
- Módulo *cn2026contract*, contendo o contrato *Protobuf* e as classes *gRPC* geradas;
- Ter permissão para interagir com os *Managed Instance Groups*, permitindo executar os comandos *Google Cloud* para o redimensionamento inicial.

7. Validação dos Resultados e Objetivos Atingidos

7.1. Testes funcionais

Como visto na figura 1, assim que a aplicação cliente é executada, é feito o pedido à Cloud Function e são retornados os endereços IP das VMs de servidor ativas no momento.

```
AVAILABLE SERVERS
1 - 35.189.222.111
Choose server: |
```

Fig.01 - Endereços IP das VMs

Depois podemos escolher entre operações funcionais (SF - Serviços de Funcionalidades) ou de gestão de elasticidade (SG - Serviços de Gestão). Estes são mostrados nas figuras 2 e 3.

```
SF MENU
1 - submit image
2 - get image data
3 - search images
4 - list images
5 - download image
0 - back
```

Fig.02 - Menu SF

```
SG MENU
1 - increase gRPC servers
2 - decrease gRPC servers
3 - increase processing instances
4 - decrease processing instances
0 - back
```

Fig.03 - Menu SG

Nos SF, temos 5 operações. Começando pela primeira, na submissão de imagem, é pedido o *path* completo da mesma, e posteriormente é feito então o *submit* para o *bucket*. É também guardada esta informação inicial no *Firestore*. Este *submit* devolve o *request_id*, para este poder ser usado posteriormente em pesquisas, como demonstrado na figura 4.

```
1
Image path:
C:\ISEL\6_semestre\CN\trab_final\cn2026client\img-04.jpg
submitImage completed.
request_id=15071f31-c438-4d33-b80b-07d72bd2d31a
```

Fig.04 - Submit Image

A operação “*get image data*” faz então o processamento para obtenção e tradução das *labels*. É pedido o *request_id*, funcionando também com o nome da imagem, e retornadas as *labels* com as respetivas pontuações de confiança, como se pode observar na figura 5.

Foi também implementada a operação “*search images*”. Esta procura imagens que tenham uma *label* específica, entre 2 datas. Se não inseridas, procura desde o início do serviço “*Timestamp*” (01/01/1970) até ao presente. Esta operação é demonstrada na figura 6.

```
2
Request ID ou nome da imagem:
15071f31-c438-4d33-b80b-07d72bd2d31a
Image data retrieved:
Request ID: 15071f31-c438-4d33-b80b-07d72bd2d31a
Image Name: img-04.jpg
Processed At: 26-05-2026 16:54:36
Labels:
- Flower -> Flor (confidence: 0.9910115)
- Petal -> Pétala (confidence: 0.981422)
- Yellow -> Amarelo (confidence: 0.97447455)
- Orange -> Laranja (confidence: 0.8810232)
- Close-up -> Fechar-se (confidence: 0.87002355)
- Flowering plant -> Planta com flor (confidence: 0.839212)
- Pollen -> Pólen (confidence: 0.8331694)
- Sulfur cosmos -> Cosmos de enxofre (confidence: 0.820777)
- Ulam raja -> Ulam raja (confidence: 0.8137131)
- Wildflower -> flor silvestre (confidence: 0.76978534)
```

Fig.05 - *Get Image Data*

```
3
Start date (dd/MM/yyyy HH:mm:ss):

End date (dd/MM/yyyy HH:mm:ss):

Label:
Pólen

Search results:
- img-04.jpg
```

Fig.06 - *Search Images*

As operações “*list images*” e “*download images*” são responsáveis por listar todas as imagens no bucket, e transferir uma delas, especificada pelo cliente, como mostra nas figuras 7 e 8, respetivamente.

```
4

Images in bucket:
- img-04.jpg
- img-01.jpg
- img-02.jpg
- img-03.jpg
```

Fig.07 - *List Images*

```
5
Request ID ou nome da imagem:
img-04.jpg
Download completed: C:\Users\jbigo\Downloads\img-04.jpg
```

Fig.08 - *Download Image*

7.2. Testes de gestão de elasticidade

Nas operações de gestão de elasticidade, temos 4 operações, mas na realidade são apenas 2, sendo estas aumento e redução de instâncias de Server ou LabelsApp. Estas são feitas a partir do menu “SG MENU”. De forma a não repetir resultados, vai ser apresentado apenas 1 exemplo de aumento. Já na redução, temos 2 casos: redução “normal”, ou redução de forma a encerrar todas as instâncias. Todas elas funcionam de forma semelhante: selecionando a operação pretendida, é pedido ao cliente qual é a quantidade de instâncias que quer aumentar ou reduzir. De seguida são mostrados os exemplos nas figuras 9, 10 e 11.

```
3
Number of processing instances to add:
2
Response:
  Success: true
  Previous size: 1
  New size: 3
  Message: Successfully increased processing instances from 1 to 3
```

☐	Status	Name ↑	Instances
☐		cn2026-labels-mig	1 → 3

Fig.09 - Increase MIG instances and GCP confirmation

```
4
Number of processing instances to remove:
2
Response:
  Success: true
  Previous size: 3
  New size: 1
  Message: Successfully decreased processing instances from 3 to 1
```

☐	Status	Name ↑	Instances
☐		cn2026-labels-mig	3 → 1

Fig.10 - Decrease MIG instances and GCP confirmation

```
2
Number of servers to remove:
4
CONFIRMATION REQUIRED: This reduction will close the gRPC server (reducing from 4 to 0).
Proceed? [y/n]: y
Confirmed. Reducing gRPC servers...
Response:
  Success: true
  Previous size: 4
  New size: 0
  Message: Successfully decreased gRPC servers from 4 to 0
```

☐	Status	Name ↑	Instances
☐		cn2026-server-mig	4 → 0

Fig.11 - Decrease MIG instances to 0 and GCP confirmation

Realça-se que quando tentamos reduzir as instâncias para 0, o servidor será encerrado. Por essa razão, é pedida uma confirmação.

8. Conclusão

O presente trabalho permitiu desenvolver um sistema distribuído na *Google Cloud Platform (GCP)* para submissão, processamento e consulta de imagens, utilizando uma arquitetura baseada em serviços independentes e comunicação assíncrona.

A solução implementada integra múltiplos serviços *GCP*, incluindo *Cloud Storage*, *Firestore*, *Pub/Sub*, *Compute Engine* e *Cloud Run Functions*, permitindo desacoplar as diferentes fases do processamento e garantir maior escalabilidade e tolerância a falhas. A utilização de *gRPC* e *Protobuf* possibilitou ainda uma comunicação eficiente entre os vários componentes do sistema.

Os objetivos definidos inicialmente foram atingidos, nomeadamente:

- submissão e armazenamento de imagens;
- processamento assíncrono com deteção e tradução de *labels*;
- consulta e pesquisa de resultados;
- gestão dinâmica da elasticidade através de operações dedicadas.

A separação entre servidores *gRPC* e *workers* de processamento em *Managed Instance Groups* distintos permitiu adaptar independentemente cada componente ao respetivo perfil de carga, melhorando a flexibilidade e eficiência da infraestrutura.

Como trabalho futuro, poderiam ser adicionadas funcionalidades como *auto-scaling* automático baseado em métricas de utilização, autenticação de utilizadores, interface gráfica *web* e suporte para novos tipos de análise de imagem.